EMBEDDED SYSTEMS & INTERNET OF THINGS (IoT) INTERNSHIP

Conducted By:

MAINCRAFTS TECHNOLOGY

Task-1 Report

Introduction to Embedded Systems & IoT Device Design

Project Title:

Smart Temperature Monitoring System

Submitted To:

Maincrafts
Embedded Systems &

IoT Internship Program

Submitted By:

Ajitesh Sharma
BTech. (EC)

Submission Date: June 2026

Table of Contents

Objective

The objective of this task is to gain a fundamental understanding of Embedded Systems and Internet of Things (IoT) technologies by studying their architecture, components, and practical applications. Embedded systems play a crucial role in modern electronic devices, enabling automation, monitoring, and intelligent decision-making across various industries. Through this task, the concepts of microcontrollers, sensors, and embedded programming are explored to understand how hardware and software interact to perform dedicated functions.

In addition to theoretical research, the task aims to provide hands-on experience in designing and simulating an embedded system project. A Smart Temperature Monitoring System has been developed using Arduino UNO, a TMP36 temperature sensor, and an RGB LED. The project demonstrates how sensor data can be collected, processed, and used to generate meaningful outputs in real time. This practical implementation helps in understanding the workflow of embedded systems and their significance in developing smart and efficient solutions for real-world applications.

Introduction to Embedded Systems

An embedded system is a specialized computing system designed to perform a specific function or a set of related functions within a larger device or machine. Unlike general-purpose computers, which are capable of executing a wide variety of tasks, embedded systems are dedicated to particular operations and are optimized for efficiency, reliability, and performance. These systems consist of both hardware and software components working together to achieve a predefined objective.

The hardware of an embedded system typically includes a microcontroller or microprocessor, memory, sensors, actuators, and communication interfaces. The software, often referred to as firmware, is programmed to control the hardware and execute specific tasks. Embedded systems are widely used because of their compact size, low power consumption, cost-effectiveness, and ability to perform real-time operations.

Today, embedded systems are present in almost every aspect of daily life. Household appliances such as washing machines, microwave ovens, and air conditioners use embedded controllers to automate their functions. In the automotive sector, embedded systems are used in engine control units, anti-lock braking systems (ABS), and airbag deployment systems. Healthcare devices such as heart rate monitors, glucose meters, and patient monitoring systems rely on embedded technology to provide accurate and reliable results. Similarly, industrial automation, robotics, consumer electronics, and IoT devices depend heavily on embedded systems for intelligent operation and data processing.

With the rapid growth of IoT and smart technologies, embedded systems have become even more important. They serve as the foundation for connected devices that can sense environmental conditions, process information, and communicate with other systems. As a result, understanding

embedded systems is essential for developing modern technological solutions and innovative smart devices.

Characteristics of Embedded Systems

Embedded systems possess several unique characteristics that distinguish them from general-purpose computing systems. These characteristics make them suitable for performing dedicated tasks efficiently and reliably.

1. **Dedicated Functionality:** An embedded system is designed to perform a specific task or a limited set of related tasks. Unlike personal computers, which can run various applications, embedded systems focus on a predefined function.
2. **Real-Time-Operation:** Many embedded systems are required to respond to inputs and events within a specific time frame. Timely execution is crucial in applications such as automotive safety systems, industrial automation, and medical devices.
3. **Reliability and Stability:** Embedded systems are expected to operate continuously for long periods with minimal failure. Their software and hardware are optimized to ensure dependable performance under different operating conditions.
4. **Low Power Consumption:** embedded devices are designed to consume minimal power, making them suitable for battery-powered and portable applications such as wearable devices and remote sensors.
5. **Compact Size:** Embedded systems are often integrated into products with limited physical space. Their compact design allows them to be incorporated into a wide range of electronic devices.
6. **Cost Efficiency:** Since embedded systems are developed for specific applications, unnecessary hardware and software components are eliminated, resulting in a cost-effective solution.
7. **High Performance for Specific Tasks:** Although embedded systems may not possess the processing power of modern computers, they are highly optimized to perform their designated tasks efficiently and accurately.

Applications of Embedded Systems

Embedded systems are widely used across various industries and have become an integral part of modern technology. Their ability to perform dedicated tasks efficiently makes them suitable for a broad range of applications.

1. **Smart Home Systems:** Embedded systems are used in smart lighting, smart thermostats, home security systems, and voice-controlled devices. These systems improve convenience, energy efficiency, and automation in homes.

2. **Automotive Industry:** Modern vehicles contain numerous embedded systems that control functions such as engine management, anti-lock braking systems (ABS), airbag deployment, parking assistance, and navigation systems.
3. **Healthcare and Medical Devices:** Medical equipment such as heart rate monitors, glucose monitoring devices, blood pressure monitors, and patient monitoring systems utilize embedded systems to provide accurate and reliable healthcare services.
4. **Industrial Automation:** Factories and manufacturing plants use embedded systems in robotics, conveyor systems, programmable logic controllers (PLCs), and monitoring equipment to increase productivity and reduce human intervention.
5. **Consumer Electronics:** Devices such as televisions, digital cameras, gaming consoles, microwave ovens, washing machines, and smart watches rely on embedded systems for their operation and control.
6. **Robotics:** Embedded systems act as the control units of robots, enabling them to process sensor data, make decisions, and perform actions autonomously or under user control.
7. **Internet of Things (IoT):** IoT devices use embedded systems to collect data from sensors, process information, and communicate with other devices over networks. Smart agriculture, smart cities, environmental monitoring, and industrial IoT are common examples of IoT applications.

The growing demand for automation, connectivity, and intelligent devices has significantly increased the importance of embedded systems in today's technological landscape.

Microcontroller vs Microprocessor

Microcontrollers and microprocessors are fundamental components of modern electronic systems. Although they perform similar computational functions, they are designed for different purposes and applications. A microcontroller is primarily used in embedded systems to perform dedicated tasks, while a microprocessor is used in general-purpose computing systems that require higher processing power and flexibility.

A microcontroller integrates the CPU, memory, and input/output peripherals on a single chip, making it compact, cost-effective, and suitable for embedded applications. In contrast, a microprocessor consists mainly of a CPU and requires external memory and peripheral devices to function effectively. As a result, microprocessors are commonly used in computers, laptops, and high-performance computing devices.

Comparison Between Microcontroller and Microprocessor:

Feature	Microcontroller	Microprocessor
Definition	A compact integrated circuit containing CPU, memory, and I/O peripherals on a single chip	A processing unit that requires external memory and peripherals
Purpose	Designed for specific tasks in embedded systems	Designed for general-purpose computing
Components	CPU, RAM, ROM, Timers, I/O Ports integrated on one chip	Primarily contains only the CPU
Power Consumption	Low	High
Cost	Relatively low	Relatively high
Processing Speed	Moderate	High
Memory	Built-in memory available	External memory required
Size	Compact	Larger system size due to external components
Applications	Home appliances, IoT devices, robotics, industrial controllers	Computers, laptops, servers, workstations
Examples	Arduino UNO, ESP32, Raspberry Pi Pico	Intel Core i5, Intel Core i7, AMD Ryzen Processors

Component Study

Embedded systems rely on various hardware components to sense, process, and respond to environmental conditions. Microcontrollers act as the processing unit of the system, while sensors are responsible for collecting data from the surroundings. In this project, Arduino UNO and a temperature sensor were used to develop a Smart Temperature Monitoring System. However, several other microcontrollers and sensors are commonly used in embedded and IoT applications.

Microcontrollers

1. **Arduino UNO:** Arduino UNO is one of the most widely used microcontroller development boards for beginners and professionals. It is based on the ATmega328P microcontroller and provides digital and analog input/output pins that allow easy interfacing with sensors and actuators. Arduino UNO is popular because of its simplicity, open-source ecosystem, and

extensive community support. It is commonly used in educational projects, robotics, automation systems, and IoT applications.

2. **ESP32:** ESP32 is a powerful microcontroller developed for IoT and wireless communication applications. It includes built-in Wi-Fi and Bluetooth capabilities, enabling devices to connect to networks and exchange data without requiring additional communication modules. ESP32 offers higher processing power and more features compared to Arduino UNO, making it suitable for smart home systems, industrial monitoring, and cloud-connected IoT devices.
3. **Raspberry Pi Pico:** Raspberry Pi Pico is a low-cost microcontroller board based on the RP2040 microcontroller chip. It provides excellent performance, flexible input/output options, and support for programming languages such as C/C++ and MicroPython. Raspberry Pi Pico is widely used in embedded projects, robotics, automation, and educational applications due to its affordability and versatility.

Sensors

1. **Temperature Sensor (TMP36):** A temperature sensor is used to measure the surrounding temperature and convert it into an electrical signal that can be processed by a microcontroller. The TMP36 sensor provides analog voltage output proportional to the measured temperature. It is commonly used in weather monitoring systems, industrial automation, smart homes, and environmental monitoring applications.
2. **Motion Sensor (PIR Sensor):** A Passive Infrared (PIR) sensor detects movement by sensing changes in infrared radiation emitted by living beings and objects. When motion is detected, the sensor generates an output signal that can be processed by a microcontroller. PIR sensors are widely used in security systems, automatic lighting systems, and smart surveillance applications.
3. **Light Sensor (LDR):** A Light Dependent Resistor (LDR) is a sensor whose resistance changes according to the intensity of light falling on its surface. As light intensity increases, the resistance decreases, and vice versa. LDRs are commonly used in automatic street lighting systems, brightness control applications, and light-sensitive automation projects.
4. These microcontrollers and sensors form the foundation of many embedded systems and IoT applications by enabling data collection, processing, and intelligent decision-making.

Smart Temperature Monitoring System

Project Objective

The objective of this project is to design and simulate a Smart Temperature Monitoring System using Arduino UNO, a TMP36 temperature sensor, and an RGB LED. The system continuously monitors the surrounding temperature and provides visual feedback based on predefined temperature ranges. By using different LED colors, the system allows users to quickly identify whether the environment is cold, normal, or hot without the need for additional display devices.

The project demonstrates the practical implementation of an embedded system by integrating a sensor, a microcontroller, and output indicators. It also provides an understanding of sensor interfacing, analog-to-digital conversion, decision-making logic, and real-time monitoring, which are essential concepts in embedded systems and IoT development.

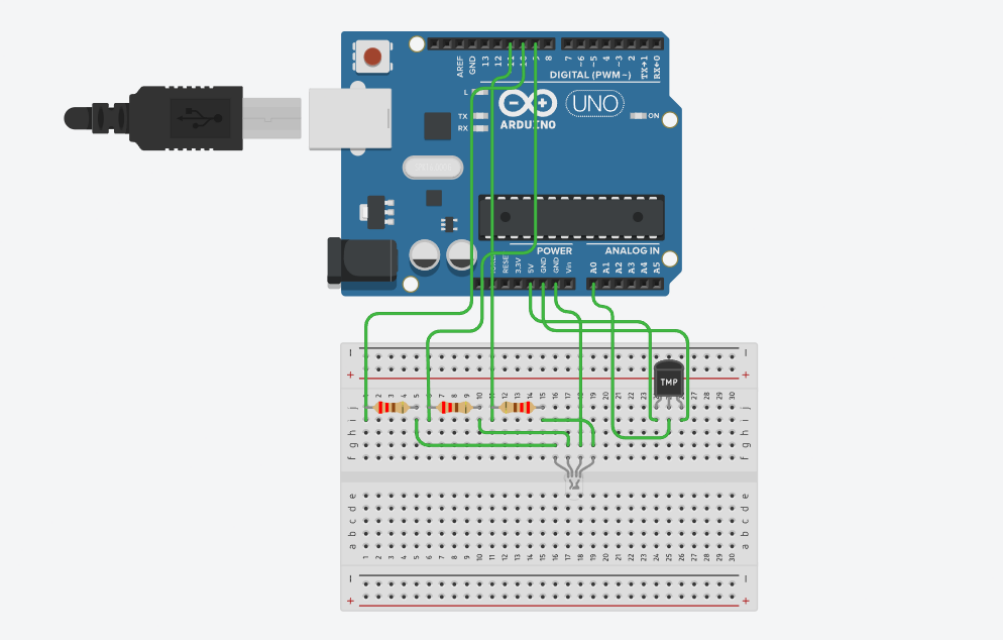


Figure 1: Smart Temperature Monitoring System

Components Used

S. No.	Component	Quantity	Purpose
1	Arduino UNO	1	Processes sensor data and controls output
2	TMP36 Temperature Sensor	1	Measures ambient temperature
3	RGB LED	1	Provides visual temperature indication
4	220Ω Resistors	3	Limits current to RGB LED
5	Jumper Wires	As Required	Establish electrical connections

Circuit Diagram

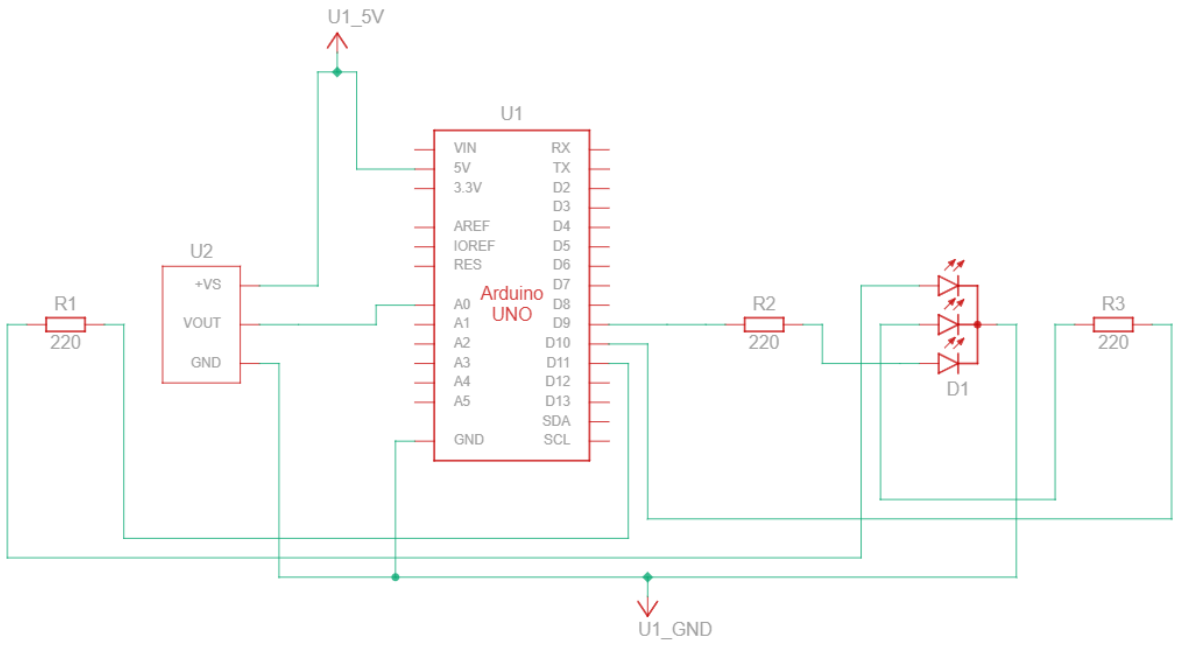


Figure 2: Smart Temperature Monitoring System Circuit Diagram

Working Principle

The Smart Temperature Monitoring System operates by continuously measuring ambient temperature using the TMP36 temperature sensor. The sensor generates an analog voltage proportional to the temperature of the surrounding environment. This analog signal is sent to the Arduino UNO through one of its analog input pins.

The Arduino reads the sensor value and converts it into a temperature value in degrees Celsius. Based on the measured temperature, the microcontroller determines the current environmental condition and activates the appropriate color of the RGB LED.

The temperature ranges used in this project are as follows:

• Temperature less than or equal to 15°C – Blue LED (Cold Condition)

• Temperature between 16°C and 30°C – Green LED (Normal Condition)

• Temperature greater than 30°C – Red LED (Hot Condition)

This color-based indication system provides an easy and effective way to monitor temperature conditions in real time. Users can identify the temperature status at a glance without requiring additional displays or monitoring devices. The project demonstrates how sensors and microcontrollers can work together to collect environmental data, process it, and generate meaningful outputs.

Arduino Program

```
int tempPin = A0;


int redPin = 11;

int greenPin = 10;

int bluePin = 9;


void setup() {

  pinMode(redPin, OUTPUT);

  pinMode(greenPin, OUTPUT);

  pinMode(bluePin, OUTPUT);


  Serial.begin(9600);
```

```
}

void loop() {

  int sensorValue = analogRead(tempPin);

  float voltage = sensorValue * (5.0 / 1023.0);
  float temperature = (voltage - 0.5) * 100;

  Serial.print("Temperature: ");
  Serial.println(temperature);

  if (temperature < 15) {
    digitalWrite(redPin, LOW);
    digitalWrite(greenPin, LOW);
    digitalWrite(bluePin, HIGH);
  }
  else if (temperature <= 30) {
    digitalWrite(redPin, LOW);
    digitalWrite(greenPin, HIGH);
    digitalWrite(bluePin, LOW);
  }
  else {
    digitalWrite(redPin, HIGH);
    digitalWrite(greenPin, LOW);
    digitalWrite(bluePin, LOW);
  }
  delay(500);
}
```

Code Explanation

The Arduino program is responsible for reading temperature values from the TMP36 sensor, processing the data, and controlling the RGB LED based on predefined temperature ranges.

Variable Declaration

The program begins by declaring variables for the temperature sensor and RGB LED pins. These variables help the Arduino identify which pins are connected to the sensor and the individual red, green, and blue terminals of the RGB LED.

Setup Function

The setup() function executes once when the Arduino is powered on or reset. In this function, the RGB LED pins are configured as output pins using the pinMode() function. Serial communication is also initialized using Serial.begin(9600) to display temperature readings in the Serial Monitor.

Reading Sensor Data

Inside the loop() function, the Arduino continuously reads analog values from the TMP36 temperature sensor using the analogRead() function. The analog value is then converted into voltage and subsequently into temperature in degrees Celsius using the sensor's conversion formula.

Temperature Processing

The measured temperature is compared with predefined threshold values to determine the environmental condition. Conditional statements (if, else if, and else) are used to categorize the temperature into cold, normal, or hot ranges.

RGB LED Control

Based on the temperature category, the Arduino activates the appropriate color of the RGB LED:

- Blue LED for temperatures less than or equal to 15°C.
- Green LED for temperatures between 16°C and 30°C.
- Red LED for temperatures greater than 30°C.

Continuous Monitoring

The entire process repeats continuously inside the loop() function, allowing real-time monitoring of temperature changes and immediate visual indication through the RGB LED.

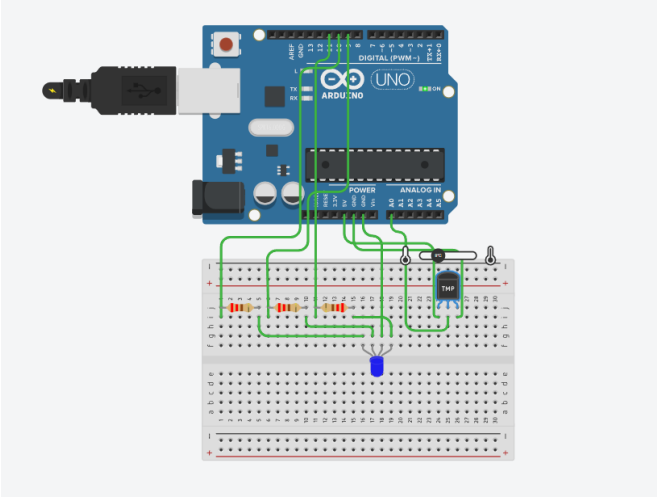
Output and Results

The Smart Temperature Monitoring System was successfully simulated using Tinkercad. The TMP36 temperature sensor accurately measured the ambient temperature, and the Arduino UNO processed the sensor data to control the RGB LED according to predefined temperature ranges.

The system produced the following outputs:

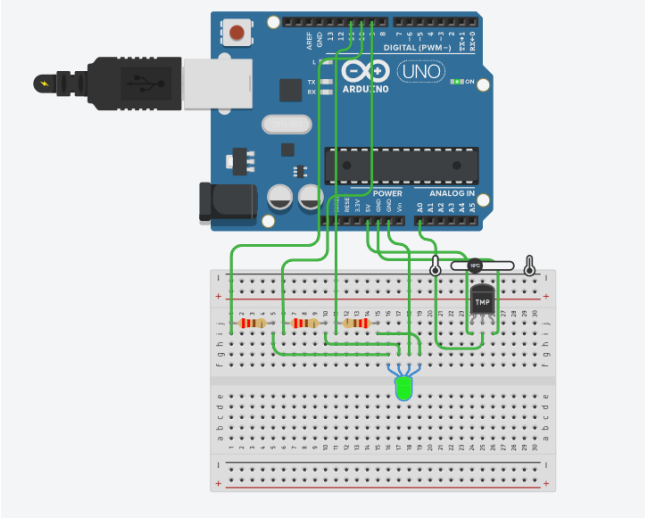
Cold Condition (≤ 15°C)

When the temperature was set to 15°C or below, the Blue LED was activated, indicating a cold environmental condition.



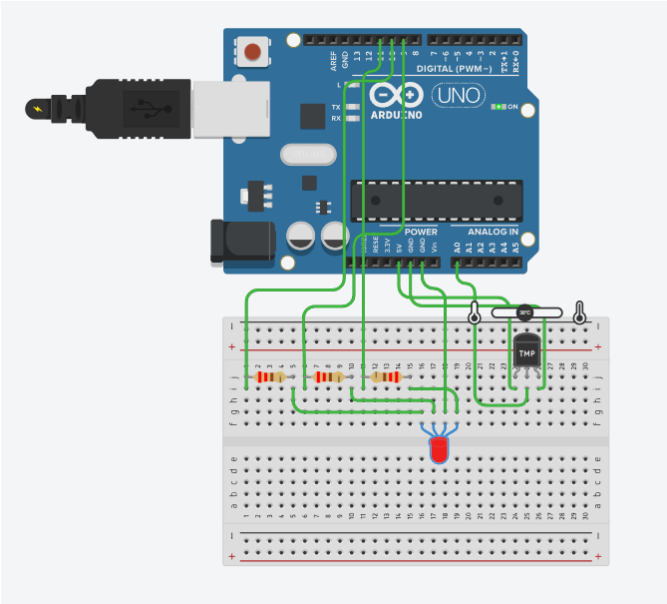
Normal Condition (16°C – 30°C)

When the temperature was maintained between 16°C and 30°C, the Green LED was activated, indicating a normal environmental condition.



Hot Condition (> 30°C)

When the temperature exceeded 30°C, the Red LED was activated, indicating a high-temperature condition.



The simulation results confirmed that the system correctly classified temperature values and provided appropriate visual feedback through the RGB LED.

Real-World Applications

The Smart Temperature Monitoring System can be applied in various real-world scenarios where temperature monitoring is important.

1. **Smart Homes:** The system can be used to monitor room temperature and provide quick visual indications of environmental conditions.
2. **Industrial Monitoring:** Factories and manufacturing plants can use temperature monitoring systems to detect overheating equipment and prevent operational failures.
3. **Data Centers and Server Rooms:** Temperature monitoring helps maintain safe operating conditions for servers and electronic equipment.
4. **Greenhouses:** Farmers can monitor environmental conditions to ensure optimal growth conditions for plants.
5. **Weather Monitoring Systems:** The project can serve as a basic environmental monitoring unit for collecting and displaying temperature information.
6. **Educational and Training Applications:** The system provides an excellent platform for learning sensor interfacing, embedded programming, and IoT concepts.

Future Enhancements

Although the Smart Temperature Monitoring System successfully performs temperature monitoring and visual indication, several enhancements can be implemented to improve its functionality and usability.

LCD Display Integration: A 16×2 LCD display can be added to show real-time temperature values and system status directly to the user.

IoT Connectivity: Wireless communication modules such as ESP32 or Wi-Fi modules can be integrated to transmit temperature data to cloud platforms for remote monitoring.

Mobile Notifications: The system can be connected to a mobile application to send alerts whenever the temperature exceeds a predefined threshold.

Data Logging: Temperature readings can be stored in a database or memory card for future analysis and record keeping.

Buzzer-Based Alert System: An audible alarm can be added to notify users when the temperature reaches critical levels.

Multi-Sensor Monitoring: Additional sensors such as humidity, gas, or light sensors can be integrated to create a complete environmental monitoring system.

These enhancements would increase the practicality and scalability of the project, making it suitable for advanced IoT and industrial applications.

Conclusion

This task provided valuable knowledge about embedded systems, microcontrollers, sensors, and their applications in modern technology. Through the research section, important concepts such as embedded systems, microcontrollers, microprocessors, and commonly used sensors were studied and analyzed.

As part of the practical implementation, a Smart Temperature Monitoring System was successfully designed and simulated using Arduino UNO, a TMP36 temperature sensor, and an RGB LED. The system continuously monitored ambient temperature and provided visual indications based on predefined temperature ranges. Blue light represented cold conditions, green light indicated normal conditions, and red light represented high-temperature conditions.

The project demonstrated the integration of hardware and software components to perform real-time monitoring and decision-making. It also enhanced understanding of sensor interfacing, embedded programming, analog signal processing, and output control. Overall, the task served as an effective introduction to embedded systems and IoT device development while providing hands-on experience in designing a practical embedded solution.